

St. Vincent Pallotti College of Engineering and Technology, Nagpur

Department of Computer Science and Engineering (Data Science) ACTIVITY REPORT 2025-2026

Title of Project: Contact Management System Using Linked List

Student Name: Aaron Geevarghese Mathews, Ann Leo **Roll No:** 24006052, 24006054

Sem: 4th Semester, 2nd Year

Date of Submission: 28-04-2026

Objective

The Contact Management System is a console-based application developed using the C programming language. The primary objective is to efficiently store, manage, and manipulate contact information such as names and phone numbers. It demonstrates practical implementation of data structures specifically the singly linked list to handle dynamic data. The system also integrates file handling to ensure data persistence across sessions.

Methodology / Algorithm

- Initialize the program and load existing contacts from contacts.txt into a singly linked list.
- Display a menu-driven interface for user interaction (Add, View, Search, Update, Delete, Sort, Stats).
- For insertion: dynamically allocate memory for a new node, store name and phone number, and prepend to the list. Check for duplicates before inserting.
- For traversal: traverse from the head node and display each contact's details.
- For search: traverse the list and compare by exact/partial name or phone number.
- For deletion: locate the target node, update the previous node's pointer, and free the deleted node's memory.
- For sorting: apply alphabetical comparison to reorder nodes (ascending/descending).
- Save all changes back to contacts.txt after every modification to ensure persistence.

Tools & Technologies Used

- Language: C Programming
- Data Structure: Singly Linked List
- File Handling: Plain-text file (contacts.txt)
- Memory Management: Dynamic allocation using malloc() and free()
- IDE/Compiler: GCC / any standard C compiler
- UI: Console/Terminal with colored output and loading animation

Main Concept of the Project

The core concept is using a Singly Linked List as the primary data structure to dynamically store contact records. Each node holds a contact's name, phone number, and a pointer to the next node. This allows flexible insertion and deletion without memory wastage, unlike fixed-size arrays. File handling (contacts.txt) is integrated to persist data across sessions. The system supports full CRUD operations, partial-name search, duplicate prevention, alphabetical sorting, and contact statistics — all through a menu-driven console interface.



Features of the Project

Feature	Description
Menu-driven Interface	User-friendly, easy to navigate
Dynamic Data Handling	Linked list for flexible memory management
File Storage	Permanent data via contacts.txt
Duplicate Prevention	Prevents adding duplicate contacts
Partial Search	Search by partial name or phone number
Sorting	Ascending & Descending alphabetical order
Statistics	Displays total contact count
Colored UI	Loading animation and colored terminal output

Advantages

Advantage	Details
Efficient Memory Usage	Only allocates memory for actual contacts
Easy Insertion & Deletion	No shifting of elements like in arrays
Scalable System	Can handle any number of contacts dynamically
Readable Logic	Simple and understandable implementation

Limitations

Limitation	Impact
No Graphical User Interface	Command-line only; no GUI interaction
Linear Search Performance	$O(n)$ time complexity for search operations
Not Suitable for Large Datasets	Performance degrades with thousands of records

Output

Menu-driven terminal interface showing options: Add Contact, View All, Search, Update, Delete, Sort, Statistics, Exit.

Sample: Adding 'John Doe, 9876543210' → 'Contact added successfully!' Viewing all contacts lists each entry with name and phone number, total count shown in statistics.

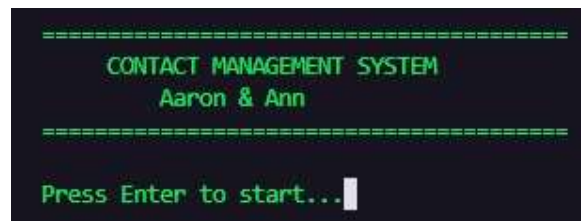


Figure 1: Startup Screen - Welcome screen displayed when the program is launched

```
=====
CONTACT MANAGEMENT SYSTEM
=====
Total Contacts: 3
=====
1. Manage Contacts
2. Advanced Search
3. View All Contacts
4. Stats
5. Exit
Enter choice: 1
```

Figure 2: Main Menu - Shows total contacts and navigation options

```
=====
CONTACT OPERATIONS
=====
1. Add Contact
2. Delete Contact
3. Update Contact
4. Back
Enter choice: 
```

Figure 3: Contact Operations Menu - Add, Delete, Update options

```
===== ADD CONTACT =====
Enter Name : Aaron Mathews
Enter Phone : 9953428577

Processing...
Contact Added!

Press Enter to continue...
```

Figure 4: Add Contact - Input and confirmation of new contact entry

```
===== DELETE CONTACT =====
Enter partial name: Aaron

===== MATCHES =====
1. Aaron Mathews (9953428577)
Select contact: 1
Confirm delete (y/n): y

Processing...
Deleted

Press Enter to continue...
```

Figure 5: Delete Contact - Search, select and confirm deletion

```
===== UPDATE CONTACT =====  
Enter partial name: Aaron  
  
❗ No matching contacts!  
  
Press Enter to continue...|
```

Figure 6: Update Contact - No matching contact found scenario

```
===== ADVANCED SEARCH =====  
1. Exact Name  
2. Partial Name  
3. Phone Number  
4. Back  
Choice: |
```

Figure 7: Advanced Search - Search type selection menu

```
===== SORT OPTIONS =====  
1. Sort A-Z  
2. Sort Z-A  
3. No Sort  
Choice: 1|
```

Figure 8: Sort Options - Alphabetical sort direction selection

```
===== CONTACT LIST =====  
  
[1]  
Name : Ann Leo  
Phone : 9764077520  
  
[2]  
Name : Annannma  
Phone : 9764077520  
  
[3]  
Name : Nikka  
Phone : 9517534862  
  
Total Contacts: 3  
  
Press Enter to continue...|
```

Figure 9: Contact List - All contacts displayed with details

```
==== STATS ====  
Total Contacts: 3  
Latest Contact: Ann Leo  
  
Press Enter to continue...
```

Figure 10: Stats - Total contact count and latest contact

Result

The Contact Management System was successfully implemented in C using a Singly Linked List. It performs all CRUD operations with file persistence, supports partial search and alphabetical sorting, and prevents duplicate entries demonstrating effective use of data structures and memory management.

Faculty Coordinator:

Prof Kalyani Satone

Dr Nischal Puri